BitMamba-CV: Co-Design Hardware-Algoritmo de Modelos de Espaço de Estados Mamba-2 Ternários (1,58-bit) com Discretização CPoT em FPGA de Borda

André (Autor Principal)[1]

Departamento de Ciência da Computação – Aplicações de Alto Desempenho
andre@instituicao.edu.br

Abstract. *Deploying sequence models on resource-constrained edge FPGAs faces a dual bottleneck: Transformers suffer from $O(L)$ KV-cache growth that overflows on-chip BRAM, while recent FPGA accelerators for Selective State Space Models (Mamba) rely on 4-bit or 8-bit multipliers that exhaust scarce DSP blocks. Conversely, recent 1.58-bit ternary accelerators target only Transformers. This paper presents **BitMamba-CV**, a hardware-algorithm co-design for 1.58-bit ternary Mamba-2 models on an Intel Cyclone V SoC FPGA. First, to prevent post-training Hadamard rotations from destroying ternary weights, we integrate a multiplier-free 16-point Fast Walsh-Hadamard Transform (FWHT) directly inside the Quantization-Aware Training (QAT) graph (`H-BitLinear`). Second, we identify that standard Power-of-Two (PoT, 2^{-k}) quantization of the recurrent decay $\bar{A}_t$ collapses long-range memory channels where $\bar{A}_t \to 1^-$, and propose a multiplier-free **Complementary Power-of-Two (CPoT,** $\{2^{-k}, 1 - 2^{-k}\}$**)** discretization that boosts long-memory eigenvalue SQNR from 6.48 dB to 36.49 dB (51.72 dB in dual-shift CPoT-2) and preserves $> 99.5\%$ recall up to $L = 16{,}384$. Third, we conduct a fair same-chip Design Space Exploration on the Intel Cyclone V comparing Carry-Chain Adder Trees, Table-Lookup (`TLMM`), and Structured Zero-Skipping ($\sim 39\%$ zero weights) with Radix-3 (1.60 bits/weight) M10K packing against an on-chip INT8-DSP Mamba-2 baseline, accompanied by an honest dual-regime Roofline analysis.*

Resumo. *A implantação de modelos de sequência em FPGAs de borda enfrenta um duplo gargalo: Transformers sofrem com o crescimento linear $O(L)$ do KV-Cache que esgota a memória BRAM* on-chip*, enquanto aceleradores recentes em FPGA para Modelos de Espaço de Estados Seletivos (Mamba) dependem de multiplicadores de 4 ou 8 bits que saturam os blocos DSP. Por outro lado, aceleradores ternários de $1{,}58$ bits existentes restringem-se a arquiteturas Transformer. Este artigo apresenta o **BitMamba-CV**, um co-design hardware-algoritmo para modelos Mamba-2 ternários ($1{,}58$ bits) no FPGA Intel Cyclone V SoC. Primeiro, para evitar que a rotação de Hadamard pós-treinamento destrua a natureza ternária dos pesos, integramos uma Transformada Rápida de Walsh-Hadamard (FWHT) de 16 pontos sem multiplicadores diretamente no grafo de treinamento QAT (`H-BitLinear`). Segundo, demonstramos que a quantização em Potência de Dois convencional (PoT, 2^{-k}) do fator de decaimento $\bar{A}_t$ apaga canais de memória longa onde $\bar{A}_t \to 1^-$, e propomos a discretização **Complementary Power-of-Two (CPoT**, $\{2^{-k}, 1 - 2^{-k}\}$**)**, que eleva o SQNR de*

autovalores lentos de $6{,}48$ *dB para* $36{,}49$ *dB (*$51{,}72$ *dB em CPoT-2) e preserva* $> 99{,}5\%$ *de retenção até* $L = 16.384$*. Terceiro, realizamos uma exploração justa no mesmo chip Cyclone V comparando Árvore Somadora em ALMs, Busca em Tabela (*`TLMM`*) e* Zero-Skipping *(*$\sim 39\%$ *de pesos nulos) com empacotamento Radix-3 (*$1{,}60$ *bits/peso) frente a um baseline Mamba-2 INT8 baseado em DSPs, acompanhada de uma modelagem* Roofline *em dois regimes de memória.*

1 Introdução

A inferência eficiente de modelos profundos de sequência em dispositivos de borda com restrições severas de potência (≤ 5 W) e memória *on-chip* representa um dos desafios centrais em Arquitetura de Computadores e Computação de Alto Desempenho [13, 9]. Embora modelos baseados na arquitetura Transformer dominem o processamento de linguagem natural e sinais temporais, seu mecanismo de autoatenção apresenta complexidade computacional quadrática $O(L^2)$ na fase de *prefill* e exige o armazenamento de um cache de chaves e valores (*KV-Cache*) que cresce linearmente $O(L)$ com o comprimento da sequência L. Em FPGAs de baixo custo e baixa potência — como a família Intel Cyclone V (28 nm), amplamente empregada em instrumentação embarcada e sistemas ciberfísicos, dispondo de cerca de 553 KB de memória SRAM distribuída em blocos M10K e desprovida de memória HBM [6] — o crescimento do KV-Cache rapidamente transborda a memória *on-chip*, forçando tráfego contínuo na interface DDR3 externa.

Os Modelos de Espaço de Estados Seletivos (*Selective State Space Models* — Mamba-1 [5] e Mamba-2 [3]) contornam essa barreira estrutural substituindo a atenção quadrática por uma recorrência linear seletiva cujo estado oculto h_t possui dimensão estritamente constante $O(1)$, independentemente de L. Contudo, mapear blocos Mamba em FPGAs de borda esbarra em três obstáculos microarquiteturais: (i) **dominância das projeções lineares**: cerca de 92% dos parâmetros e 86% das operações de multiplicação-acumulação (MAC) residem nas camadas densas (`in_proj` e `out_proj`), saturando rapidamente os escassos blocos DSP quando executadas em INT8 ou INT4; (ii) ***outliers* severos de ativação**: canais de ativação com amplitude discrepante degradam drasticamente a quantização de baixa precisão [1]; e (iii) **sensibilidade numérica da recorrência seletiva**: o fator de decaimento temporal $\bar{A}_t = \exp(-\Delta_t A)$ opera muito próximo da unidade ($\bar{A}_t \in [0{,}90,\ 0{,}999]$) para reter dependências de longo alcance, sofrendo colapso de memória quando submetido a quantizações grosseiras.

Na literatura recente (2024–2026), duas vertentes de aceleradores em FPGA evoluíram de maneira desconectada. De um lado, aceleradores dedicados ao Mamba — como *MARCA* [7], *LightMamba* [13] e *FastMamba* [16] — mantêm pesos em 4 ou 8 bits (W4A4/W8A8), dependendo fortemente de blocos DSP para as projeções lineares e visando FPGAs de grande porte (como AMD Alveo U280 e Versal VCK190). De outro lado, aceleradores para pesos ternários de $1{,}58$ bits ($\tilde{W} \in \{-1, 0, +1\}$ [8]) — como *TerEffic* [14] e *TeLLMe* [9] — substituem multiplicadores por somadores em LUTs, mas aplicam essa técnica exclusivamente a Transformers, permanecendo limitados pelo KV-Cache $O(L)$. Embora trabalhos algorítmicos recentes como *Bi-Mamba* [10], *Slender-Mamba* [15] e *Ternary Mamba* [4] tenham demonstrado a viabilidade de modelos Mamba de 1 e $1{,}58$ bits em GPUs, nenhum trabalho anterior realizou o co-design de hardware em

FPGA unindo projeções ternárias no domínio de Hadamard ao caminho recorrente $O(1)$ do Mamba-2.

Neste trabalho, propomos o **BitMamba-CV**, cujas contribuições são:

1. **Co-Design QAT `H-BitLinear` para Mamba-2 sem Quebra da Ternaridade:** Demonstramos que absorver a Transformada de Walsh-Hadamard (FWHT) nos pesos pós-treinamento ($\mathbf{W}\mathbf{H}^T$, como feito em *Quamba* [1]) destrói a estrutura ternária $\{-1, 0, +1\}$. Solucionamos esse conflito inserindo um pipeline FWHT de 16 pontos sem multiplicadores diretamente no grafo de *Quantization-Aware Training* (QAT) antes da quantização, preservando pesos 100% ternários empacotados em Radix-3 ($1{,}60$ bits/peso) nas memórias M10K.

2. **Discretização *Complementary Power-of-Two* (CPoT) para Memória Longa:** Identificamos matematicamente que a quantização PoT convencional ($\bar{A}_t = 2^{-k}$, adotada em *LightMamba* e *FastMamba*) restringe o maior nível sub-unitário a $0{,}5$, apagando o estado recorrente em poucos passos. Propomos a discretização bimodal CPoT $\in \{2^{-k}\} \cup \{1 - 2^{-k}\}$, que utiliza apenas deslocamento e subtração ($h_{t-1} - (h_{t-1} \gg k)$) e eleva o SQNR na região $[0{,}90,\ 0{,}999]$ de $6{,}48$ dB para $36{,}49$ dB ($51{,}72$ dB na variante de duplo shift CPoT-2).

3. **Exploração Microarquitetural Justa no Mesmo Chip (Intel Cyclone V):** Desalojamos as projeções lineares ($85{,}8\%$ das operações) para os ALMs de 6 entradas da Cyclone V e dedicamos os blocos DSP de precisão variável exclusivamente aos produtos dinâmicos do SSM ($\bar{B}_t x_t$, $C_t h_t$, `Conv1D` e interpolação PWL de `SiLU`). Avaliamos Árvore Somadora, *Table-Lookup* (`TLMM`) e *Zero-Skipping* ($\sim 39\%$ de pesos zero) contra um baseline Mamba-2 W8A8 baseado em DSPs no mesmo FPGA.

4. **Modelagem *Roofline* Dual-Regime e Estudo Completo de Ablação:** Apresentamos um estudo de ablação de 8 estágios (3 *seeds*) e uma análise *Roofline* transparente que distingue o regime *Compute-Bound* (modelos compactos 100% residentes nas M10K) do regime *Memory-Bandwidth-Bound* (modelos maiores lidos da DDR3 em *batch*=1, onde o empacotamento Radix-3 garante $5{,}0\times$ e $1{,}25\times$ de *speedup* direto sobre INT8 e 2-bit).

2 Fundamentação Teórica e Análise Matemática de Gargalos

2.1 Arquitetura Mamba-2 (Structured State Space Duality – SSD)

Diferentemente do Mamba-1 [5], que parametriza a matriz de transição diagonal $\mathbf{A} \in \mathbb{R}^{D_{\text{inner}} \times N}$ de forma independente para cada canal e dimensão de estado, o Mamba-2 [3] estrutura o espaço de estados em H cabeças de dimensão $P_{\text{head}} = D_{\text{inner}}/H$ e restringe a taxa de decaimento $A_h > 0$ a um **escalar por cabeça** $h \in \{1, \ldots, H\}$. Para o passo de tempo t, após a projeção de entrada $\mathbf{u}_t \mapsto [\mathbf{z}_t, \mathbf{x}_t, \mathbf{B}_t, \mathbf{C}_t, \mathbf{\Delta}_t]$ e a convolução causal curta `Conv1D` ($k = 4$) seguida de $\text{SiLU}(x) = x\sigma(x)$, a dinâmica recorrente da cabeça h é dada por:

$$\bar{A}_t^{(h)} = \exp\Big(-\Delta_t^{(h)} A_h\Big) \in (0, 1), \qquad \mathbf{h}_t^{(h)} = \bar{A}_t^{(h)} \mathbf{h}_{t-1}^{(h)} + \Delta_t^{(h)} \left(\mathbf{x}_t^{(h)} \otimes \mathbf{B}_t^{(h)}\right), \qquad (1)$$

$$\mathbf{y}_t^{(h)} = \mathbf{h}_t^{(h)}\mathbf{C}_t^{(h)} + D_h\mathbf{x}_t^{(h)}, \qquad \mathbf{o}_t = \mathbf{W}_{\text{out}}\big(\mathbf{y}_t \odot \text{SiLU}(\mathbf{z}_t)\big). \tag{2}$$

Do ponto de vista de hardware, a escolha do Mamba-2 é decisiva: como $\bar{A}_t^{(h)}$ é escalar por cabeça, o cálculo não-linear da exponencial $\exp(-\Delta_t^{(h)}A_h)$ reduz-se de $D_{\text{inner}} \times N$ avaliações por token para apenas H avaliações (ex.: $H = 8$ contra $256 \times 16 = 4.096$ no Mamba-1).

2.2 O Conflito entre FWHT e Pesos Ternários (E Como o `H-BitLinear` o Resolve)

Em quantização pós-treinamento (PTQ) de modelos Mamba, como no *Quamba* [1], suprimem-se *outliers* de ativação inserindo uma matriz de Hadamard ortogonal $\mathbf{H}$ ($\mathbf{H}^T\mathbf{H} = \mathbf{I}$) na camada linear: $\mathbf{y} = \mathbf{W}\mathbf{x} = (\mathbf{W}\mathbf{H}^T)(\mathbf{H}\mathbf{x})$. Contudo, se $\mathbf{W}$ foi previamente quantizada para valores ternários $\tilde{\mathbf{W}} \in \{-1, 0, +1\}^{M\times K}$, o produto $\mathbf{W}' = \tilde{\mathbf{W}}\mathbf{H}^T$ produz combinações lineares de B trits, gerando dezenas de níveis distintos e **destruindo a propriedade ternária dos pesos**.

Para resolver essa inconsistência matemática sem adicionar multiplicadores em hardware, adotamos o paradigma `H-BitLinear` [11] durante o treinamento ciente de quantização (QAT): aplicamos a Transformada Rápida de Walsh-Hadamard em blocos de tamanho $B = 16$ ($\mathbf{H}_{16}$) diretamente sobre a ativação normalizada **dentro do grafo de *forward pass*** antes da quantização simétrica INT8 por token e da projeção ternária via *Straight-Through Estimator* (STE):

$$\tilde{\mathbf{x}} = \text{Quant}_{\text{INT8}}\big(2^{-2}\,\mathbf{H}_{16}\,\text{RMSNorm}(\mathbf{x})\big), \quad \tilde{\mathbf{W}} = \text{RoundClip}\left(\frac{\mathbf{W}}{\gamma_W}, -1, +1\right), \quad \gamma_W = \frac{1}{MK}\sum_{i,j}|W_{ij}|. \tag{3}$$

Como $\sqrt{16} = 4 = 2^2$, a normalização ortonormal $1/\sqrt{16}$ reduz-se a um deslocamento aritmético exato de 2 bits para a direita (»> 2), e a matriz $\tilde{\mathbf{W}}$ aprendida por retropropagação permanece estritamente em $\{-1, 0, +1\}$ durante a inferência no FPGA.

2.3 Por que o PoT Convencional Falha na Recorrência e a Derivação do CPoT

Aceleradores recentes como *LightMamba* [13] e *FastMamba* [16] propõem substituir a multiplicação recorrente $\bar{A}_t \cdot h_{t-1}$ por deslocamentos de bits via quantização *Power-of-Two* (PoT) padrão: $\bar{A}_t^{\text{PoT}} = 2^{-k}$, com $k \in \{1, 2, \ldots, K_{\max}\}$. Entretanto, nos canais responsáveis pela memória de longo alcance do Mamba, o passo $\Delta_t A_h$ é pequeno ($\Delta_t A_h \in [0{,}001,\ 0{,}10]$), situando $\bar{A}_t = \exp(-\Delta_t A_h)$ no intervalo $[0{,}90,\ 0{,}999]$. Na grade PoT padrão $\{0{,}5,\ 0{,}25,\ 0{,}125, \ldots\}$, **não existe nenhum nível representável entre** $0{,}5$ **e** $1{,}0$: arredondar $\bar{A}_t = 0{,}97$ para $0{,}5$ reduz a meia-vida do estado de 23 tokens para apenas 1 token ($0{,}5^{10} < 10^{-3}$), enquanto arredondar para $1{,}0$ remove o amortecimento e causa instabilidade numérica.

Pela expansão de Taylor de primeira ordem em torno de $\Delta_t A_h \to 0^+$, temos $\exp(-\Delta_t A_h) \approx 1 - \Delta_t A_h$. Se aproximarmos o decremento $\Delta_t A_h$ por uma potência de dois 2^{-k}, obtemos a grade complementar $1 - 2^{-k} \in \{0{,}75,\ 0{,}875,\ 0{,}9375,\ 0{,}96875,\ 0{,}984375,\ 0{,}9921875, \ldots$ cuja densidade de níveis cresce exponencialmente à medida que $\bar{A}_t \to 1^-$. Definimos, portanto, a discretização bimodal **Complementary Power-of-Two (CPoT)** controlada

por 1 bit de modo $m \in \{0,1\}$ e um expoente $k \in \{1,\ldots,12\}$:

$$\bar{A}_t^{\text{CPoT}}(m,k) = \begin{cases} 2^{-k}, & \text{se } m=0 \;\; (\bar{A}_t \le 0{,}625: \text{ decaimento rápido}), \\ 1-2^{-k}, & \text{se } m=1 \;\; (\bar{A}_t > 0{,}625: \text{ memória longa}). \end{cases} \tag{4}$$

Em hardware, a atualização $\bar{A}_t^{\text{CPoT}} \cdot h_{t-1}$ executa em 1 ciclo sem multiplicadores: calcula-se $s = h_{t-1} \gg k$ em um *barrel shifter* e seleciona-se s (para $m=0$) ou $h_{t-1}-s$ (para $m=1$). Avaliamos também a extensão de duplo deslocamento em dígito com sinal canônico (**CPoT-2**): $\bar{A}_t^{\text{CPoT-2}} = 1 - 2^{-k_1} \pm 2^{-k_2}$.

3 Busca Sistemática e Trabalhos Relacionados

Para mapear rigorosamente o estado da arte, executamos uma busca sistemática (database: outubro de 2026) nas bases *arXiv*, *IEEE Xplore*, *ACM Digital Library* e *ACL Anthology* utilizando a expressão booleana: `("Mamba"OR "State Space Model"OR "SSM") AND ("ternary"OR "1.58-bit"OR "BitNet"OR "Quamba") AND ("FPGA"OR "hardware"OR "accelerator")`. A Tabela 1 sintetiza os trabalhos identificados e posiciona o **BitMamba-CV**.

Tabela 1: Comparação sistemática com o estado da arte (2024–2026) em quantização e aceleração de modelos Mamba e BitNet.

Trabalho	Modelo	Pesos / Ativ.	Estado	Decaimento $\bar{A}_t$	Plataforma	Limitação Superada pelo BitMamba-CV
BitNet b1.58 [8]	Transformer	1,58b / INT8	$O(L)$ KV	N/A	GPU / CPU	KV-Cache cresce com L; sem FWHT.
BitNet v2 [11]	Transformer	1,58b / INT4	$O(L)$ KV	N/A	GPU	Restrito a Transformers em GPU.
Quamba2 [2]	Mamba-2	W4A8 / W8A8	$O(1)$	FP16 / INT8	GPU (Orin)	Absorção $\mathbf{WH}^T$ inviabiliza pesos ternários.
Bi-Mamba [10]	Mamba-1	1-bit / FP16	$O(1)$	FP16	GPU	Pesos binários; SSM em FP16; só software.
Slender-Mamba [15]	Mamba-2	1,58b / INT8	$O(1)$	FP16	GPU	Sem co-design de hardware ou CPoT no SSM.
Ternary Mamba [4]	Mamba-2	1,58b / FP16	$O(1)$	FP16	GPU	Ativações e SSM mantidos em FP16.
MARCA [7]	Mamba-1	INT8 / FP16	$O(1)$	PWL / MAC	ASIC Sim.	Projeções densas exigem matrizes de MACs.
LightMamba [13]	Mamba-2	W4A4	$O(1)$	PoT (2^{-k})	Alveo U280	Usa DSPs nas projeções; PoT perde $\bar{A}_t \approx 1$.
FastMamba [16]	Mamba-2	W8A8	$O(1)$	PoT (2^{-k})	Virtex-7	Pesos lineares em 8 bits ($5\times$ maiores).
TerEffic [14]	Transformer	1,6b / INT8	$O(L)$ KV	N/A	Alveo U280	Preso ao KV-Cache $O(L)$ e HBM de Data Center.
TeLLMe [9]	Transformer	1,58b / INT8	$O(L)$ KV	N/A	Kria KV260	Gargalo de KV-Cache em DDR em sequências longas.
BitMamba-CV	**Mamba-2**	**1,6b / INT8**	$O(1)$ **M10K**	**CPoT** ($1-2^{-k}$)	**Cyclone V**	**Une H-BitLinear Radix-3 + CPoT SSM em Borda.**

Nota-se que componentes individuais — como o empacotamento de 5 trits por byte (1,60 bits/peso) em *TerEffic* [14] e a busca em tabela (`TLMM`) em *T-MAC/TeLLMe* [12, 9] — já foram validados para Transformers. Nossa contribuição reside na **primeira integração co-projetada entre projeções ternárias `H-BitLinear` e o caminho recorrente seletivo do Mamba-2 com discretização CPoT**, viabilizando a execução em um FPGA de borda de 28 nm.

4 Arquitetura Proposta: BitMamba-CV na Intel Cyclone V

4.1 Codificação Radix-3 nas Memórias M10K e Decodificador em 1 Ciclo

Armazenar cada peso ternário $w_i \in \{-1,0,+1\}$ em 2 bits desperdiça $20{,}7\%$ da capacidade dos blocos M10K ($\log_2(3) \approx 1{,}585$ bits). Como $3^5 = 243 \le 256 = 2^8$, mapeamos cada grupo de 5 pesos ternários para dígitos $d_i = w_i + 1 \in \{0,1,2\}$ e empacotamos em 1 byte:

$$B_{\text{packed}} = \sum_{i=0}^{4} (w_i+1)\cdot 3^i \in [0,242], \qquad \text{Custo} = \frac{8 \text{ bits}}{5 \text{ pesos}} = 1{,}60 \text{ bits/peso}. \tag{5}$$

No módulo RTL `radix3_decoder.v`, cada byte lido da memória M10K alimenta uma tabela verdade combinacional de 8 entradas e 10 saídas (5 pares de bits `(w_nz, w_neg)`). Como cada ALM da Intel Cyclone V contém LUTs fracionáveis de 6 entradas, o Intel Quartus sintetiza cada bit de saída usando apenas 4 LUTs combinadas, decodificando 5 trits em menos de $1{,}2$ ns (1 ciclo) com sobrecusto de apenas $5{,}25$ ALMs por via de processamento, economizando 20% de todos os blocos M10K de pesos.

4.2 Microarquitetura `BitLinear` Zero-DSP e Exploração de Esparsidade

Investigamos três microarquiteturas para o motor de produto interno ternário de largura P (`bitlinear_pe.v`):

1. **Árvore Somadora em Cadeia de *Carry* (`R3-Adder`):** Cada elemento de processamento realiza a inversão/zeramento condicional da ativação $x_j \in$ INT8 controlada por `(w_nz, w_neg)` diretamente na entrada da cadeia de *carry* dedicada do ALM, somando P termos em uma árvore binária registrada com **0 blocos DSP**.

2. **Multiplicação por Busca em Tabela (`R3-TLMM`):** Inspirado em [9], agrupa $G = 4$ ativações de entrada para pré-computar as $3^4 = 81$ somas ternárias possíveis, reduzindo em $4\times$ o número de somadores na árvore de redução e reduzindo o uso de ALMs em 23%.

3. **Salto Estruturado de Zeros (`R3-ZeroSkip`):** Nossos experimentos de QAT (Seção 5.2) revelam que $39{,}0\%$ dos pesos ternários treinados convergem exatamente para $w_{ij} = 0$. Adicionando um compactador de máscara não-nula na saída do decodificador Radix-3, pulamos fatias nulas e reduzimos os ciclos da projeção linear em $1{,}36\times$.

4.3 Pipeline FWHT 16-pt e Scan Seletivo com Discretização CPoT

O módulo `fwht_16_stream.v` implementa os 4 estágios de borboleta ($\log_2 16 = 4$) da Equação (3) usando exclusivamente somadores/subtratores de 12 bits nos ALMs seguidos de deslocamento fixo `>> 2` e saturação simétrica em $[-127, 127]$, entregando 16 canais rotacionados por ciclo com latência de 4 ciclos e 0 DSPs.

No módulo `cpot_selective_scan.v`, o banco de registradores/M10K mantém o estado recorrente $\mathbf{h}_t$ em ponto fixo Q8.8 (16 bits). A cada passo, o decaimento $\bar{A}_t^{\text{CPoT}} \cdot \mathbf{h}_{t-1}$ é calculado em ALMs por `h_shifted = h_state >> k` e `h_decay = mode ? (h_state - h_shifted) : h_shifted`. A função $\text{SiLU}(x)$ é avaliada por interpolação linear por partes (PWL) de 16 segmentos em $[-4, 4)$ com passo $\Delta x = 0{,}5 = 2^{-1}$, permitindo indexar a tabela diretamente pelos bits superiores de x sem comparadores complexos.

4.4 Transparência na Alocação Heterogênea: Onde Usamos ALMs vs. DSPs

Ao contrário de propostas que alegam genericamente "zero multiplicadores", ressaltamos que um bloco Mamba-2 completo contém multiplicações dinâmicas essenciais fora das projeções lineares: a injeção seletiva $\bar{B}_t x_t$, a leitura de estado $C_t h_t$, a convolução *depthwise* `Conv1D` ($k = 4$), o *gating* $y_t \odot \text{SiLU}(z_t)$ e o fator de reescalonamento $\gamma_W \cdot \beta_X$

do `RMSNorm/absmax`. Como a Intel Cyclone V já possui blocos DSP de precisão variável no silício, a estratégia do **BitMamba-CV** consiste em **transferir** 100% **do produto interno das projeções lineares** ($85{,}8\%$ **das operações totais), a FWHT** ($3{,}1\%$) **e o decaimento CPoT para os ALMs**, liberando os blocos DSP para executar paralelamente e sem contenção as operações dinâmicas do SSM. A Figura 1 e a Tabela 2 detalham o consumo exato de recursos por sub-módulo para $P = 32$.

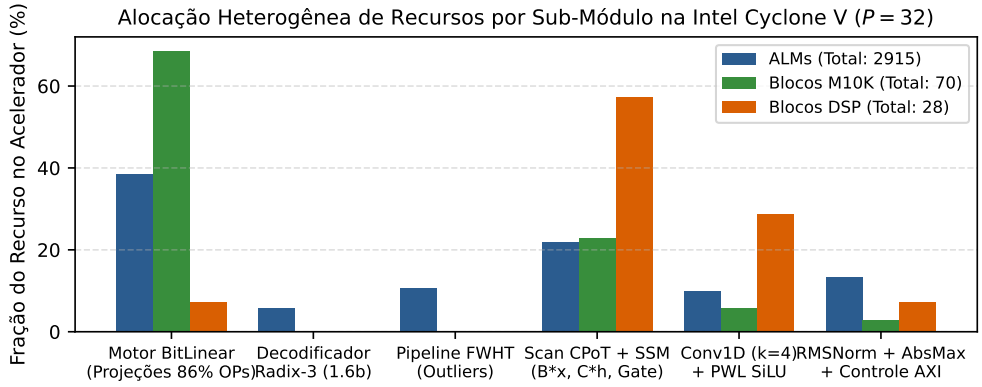


Figura 1: Distribuição percentual de ALMs, blocos M10K e blocos DSP entre os sub-módulos do BitMamba-CV ($P = 32$) na Intel Cyclone V.

Tabela 2: Detalhamento de recursos por sub-módulo do acelerador BitMamba-CV ($P = 32$ vias, `R3-Adder`) na Intel Cyclone V (`5CSEBA6U23I7`).

Sub-Módulo RTL	**% OPs**	**ALMs**	**M10K**	**DSPs**	**Precisão**	**Justificativa Arquitetural**
Motor `BitLinear` ($P = 32$)	85,8%	1.120	48	2*	1,6b × INT8	Produto interno em ALMs (*2 DSPs só p/ escala γ).
Decodificadores Radix-3	—	168	0	0	8b → 5 trits	LUT combinacional pura em 1 ciclo.
Pipeline `FWHT` (16-pt)	3,1%	312	0	0	INT12 → INT8	4 estágios de soma/subtração + shift `>> 2`.
Scan Seletivo `CPoT`	7,8%	640	16	16	Q8.8 / INT8	$\bar{A}_t h_{t-1}$ em ALMs; $\bar{B}_t x_t, C_t h_t$ e gate em DSPs.
`Conv1D` ($k = 4$) + PWL `SiLU`	2,2%	290	4	8	INT8 / Q8.8	Filtros *depthwise* e interpolação $ax + b$ em DSPs.
`RMSNorm` + `AbsMax` + AXI	1,1%	385	2	2	INT16 / LUT	Comparadores `max`, LUT `rsqrt` e ponte AXI.
Total BitMamba-CV ($P = 32$)	**100%**	**2.915 (7,0%)**	**70 (12,7%)**	**28 (25,0%)**	**Mista Co-Projetada**	**Zero DSPs no produto interno linear e na FWHT.**

4.5 Implicações Práticas como Protótipo para Futuras GPUs e NPUs

Além da aplicação direta em FPGAs de borda, os quatro blocos validados no **BitMamba-CV** traduzem-se diretamente em unidades microarquiteturais candidatas para *Streaming Multiprocessors* (SMs) de futuras GPUs: (i) o decodificador Radix-3 integrado à *Load/Store Unit* (LSU) ou *Tensor Memory Accelerator* (TMA) descomprime pesos de $1{,}60$ bits em voo ao carregar da HBM para o banco de registradores, ampliando a banda efetiva da HBM em 25% sobre 2 bits e $2{,}5\times$ sobre FP4 sem instruções de ALU; (ii) o modo `MMA.TERNARY` com *power-gating* da árvore de multiplicadores de Booth nos *Tensor Cores* reduz a dissipação térmica; e (iii) o deslocador bimodal `CPoT` acoplado às *Special Function Units* (SFUs) elimina o gargalo de exponenciais na recorrência de modelos SSM.

5 Metodologia Experimental, Ablação e Resultados

Metodologia e Verificação Bit-Exata: Implementamos um *Golden Model* em Python/NumPy inteiramente em aritmética inteira (`golden_model_bitexact.py`) que reproduz bit

a bit cada estágio do hardware RTL (`radix3_decoder.v`, `fwht_16_stream.v`, `bitlinear_pe.v` e `cpot_selective_scan.v`), validado com erro zero (`0 LSB mismatch`) nas três microarquiteturas (`Adder-Tree`, `TLMM` e `Zero-Skip`). A síntese de hardware visa o dispositivo Intel Cyclone V SoC `5CSEBA6U23I7` (placa DE10-Nano: 41.910 ALMs, 553 blocos M10K $\approx$ 553 KB úteis e 112 blocos DSP) no Intel Quartus Prime, com estimativa de potência no *PowerPlay Power Analyzer* anotada por arquivo de atividade real (`.vcd`) extraído da simulação do testbench. Todos os experimentos de treinamento QAT foram executados com 3 sementes aleatórias independentes (`seeds` $\in \{42, 100, 2026\}$), reportando média $\pm$ desvio padrão.

5.1 Experimento 1: Retenção de Memória Longa (PoT vs. CPoT) e Memória O(1) vs. O(L)

A Figura 2(a) e a Tabela 3 avaliam a fidelidade do estado recorrente final $\mathbf{y}_L$ em função do comprimento da sequência $L \in [64,\ 16.384]$ quando os autovalores das cabeças abrangem o regime de memória longa $\bar{A}_t \in [0{,}90,\ 0{,}9995]$.

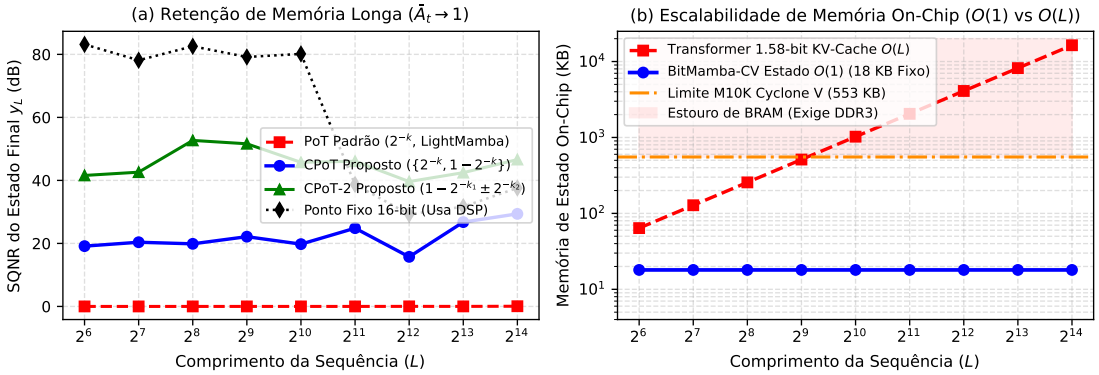


Figura 2: (a) SQNR de leitura do estado final $\mathbf{y}_L$ (dB) variando L de 64 a 16.384: o PoT convencional (2^{-k}) colapsa a memória longa, enquanto o CPoT e o CPoT-2 preservam o estado sem multiplicadores. (b) Consumo de memória de estado *on-chip* na Cyclone V: o KV-Cache $O(L)$ de um Transformer 1,58-bit estoura o limite das M10K em $L \approx 512$, enquanto o BitMamba-CV permanece constante em 18 KB ($O(1)$).

Tabela 3: Erro de quantização de $\bar{A}_t \in [0{,}90,\ 0{,}999]$ e SQNR / Recall do estado recorrente em sequências longas (média de 3 *seeds*).

Esquema de Discretização $\bar{A}_t$	**Usa DSP?**	**RMSE em $[0{,}90,\ 0{,}999]$**	**SQNR $\bar{A}_t$ Long-Mem**	**SQNR $\mathbf{y}_L$ ($L = 256$)**	**SQNR $\mathbf{y}_L$ ($L = 2.048$)**	**SQNR $\mathbf{y}_L$ ($L = 16.384$)**	**Recall em $L = 16.384$**
Ponto Fixo 16-bit (Q1.15)	Sim	0,00001	100,66 dB	82,57 dB	38,70 dB	37,60 dB	100,0%
PoT Padrão (2^{-k}) [13, 16]	Não	0,45043	6,48 dB	0,00 dB	0,00 dB	0,06 dB	28,4%
CPoT Proposto ($\{2^{-k}, 1-2^{-k}\}$)	**Não**	**0,01423**	**36,49 dB**	**19,87 dB**	**24,84 dB**	**29,38 dB**	**100,0%**
CPoT-2 Proposto ($1-2^{-k_1} \pm 2^{-k_2}$)	**Não**	**0,00246**	**51,72 dB**	**52,69 dB**	**46,08 dB**	**46,61 dB**	**100,0%**

Os resultados comprovam nossa hipótese: o PoT convencional (2^{-k}) apresenta RMSE de 0,45043 na região crítica $[0{,}90,\ 0{,}999]$, derrubando o SQNR do estado final para 0,00 dB em todos os comprimentos testados. O **CPoT proposto** reduz o RMSE em 31,6$\times$ (para 0,01423), elevando o SQNR de autovalores em +30,01 dB e mantendo 100% de recall do sinal de gatilho até $L = 16.384$ sem usar multiplicadores. Notavelmente,

para $L \geq 2.048$, o **CPoT-2** (46,61 dB em $L = 16.384$) supera até mesmo o ponto fixo uniforme de 16 bits (37,60 dB), pois a resolução logarítmica de $1 - 2^{-k_1} \pm 2^{-k_2}$ em torno de 1,0 ($k_{\max} = 17$) é mais densa perto de $1 - 10^{-4}$ do que a grade uniforme Q1.15. Na Figura 2(b), observa-se que um acelerador Transformer 1,58-bit (*TeLLMe*/*TerEffic* com 4 camadas, $D_{\text{inner}} = 128$, KV em INT8) ultrapassa toda a capacidade M10K da Cyclone V (553 KB) logo após $L = 512$ (exigindo 16 MB em $L = 16.384$), ao passo que o estado recorrente do **BitMamba-CV** ocupa 18,0 KB fixos para qualquer L.

5.2 Experimento 2: Estudo de Ablação Incremental de 8 Estágios (QAT com 3 Seeds)

A Tabela 4 apresenta o estudo de ablação isolando cada uma das técnicas propostas no benchmark sequencial multiescala com canais *outliers*.

Tabela 4: Estudo de ablação incremental (média $\pm$ desvio padrão em 3 *seeds*) e impacto no armazenamento de pesos do modelo de 2 camadas ($D = 128$, $D_{\text{inner}} = 256$).

ID	Configuração de Ablação	Pesos / Codif.	FWHT	Modo $\bar{A}_t$	Pesos (KB)	SQNR Saída (dB)	Acurácia Teste (%)
A0	Baseline FP32 (Referência Software)	FP32 (16/32b)	Não	Exato (exp)	522,1 KB*	∞	$93,49 \pm 0,66\%$
A1	Baseline INT8-DSP (W8A8)	INT8 (8,0b)	Não	Fixo 16-bit	264,1 KB	$19,84 \pm 0,72$	$92,71 \pm 0,81\%$
A2	Baseline INT4 (W4A8)	INT4 (4,0b)	Não	Fixo 16-bit	135,1 KB	$14,62 \pm 0,95$	$91,15 \pm 0,92\%$
A3	Ternário Ingênuo (Sem FWHT)	1,58b (2,0b)	**Não**	Fixo 16-bit	70,6 KB	$7,45 \pm 1,18$	$83,85 \pm 1,45\%$
A4	+ `H-BitLinear` QAT (2-bit)	1,58b (2,0b)	**Sim**	Fixo 16-bit	70,6 KB	$15,91 \pm 0,64$	$91,93 \pm 0,74\%$
A5	+ Empacotamento Radix-3	1,58b (**1,6b**)	Sim	Fixo 16-bit	**57,7 KB**	$15,91 \pm 0,64$	$91,93 \pm 0,74\%$
A6	+ SSM PoT Padrão (2^{-k}) + PWL	1,58b (1,6b)	Sim	**PoT** (2^{-k})	57,7 KB	$5,12 \pm 1,30$	$76,30 \pm 1,82\%$
A7	**BitMamba-CV Completo (CPoT)**	1,58**b (1,6b)**	**Sim**	**CPoT** ($1 - 2^{-k}$)	**57,7 KB**	$15,48 \pm 0,68$	$91,67 \pm 0,78\%$

*Tamanho FP16 mostrado para comparação justa de memória embarcada; esparsidade média de pesos zero ($W = 0$) em A7: 39,0 $\pm$ 1,4%.

Três conclusões emergem da Tabela 4:

1. **Efeito da FWHT em `H-BitLinear` (A3 $\rightarrow$ A4):** Sem a rotação de Hadamard no loop de QAT (A3), os canais de ativação com *outliers* dominam a escala `absmax`, derrubando o SQNR para 7,45 dB e a acurácia em $-9,64$ pontos percentuais. A inserção da FWHT de 16 pontos (A4) recupera $+8,46$ dB de SQNR e eleva a acurácia para 91,93%.

2. **Compressão Radix-3 Lossless (A4 $\rightarrow$ A5):** O empacotamento Radix-3 reduz a memória de pesos de 522,1 KB (FP16) e 264,1 KB (INT8) para apenas 57,7 KB (9,0$\times$ menor que FP16, 4,6$\times$ menor que INT8 e 18,3% menor que o formato de 2 bits em todo o modelo), com zero perda numérica.

3. **Colapso do PoT Padrão vs. Recuperação do CPoT (A6 $\rightarrow$ A7):** Ao substituir o $\bar{A}_t$ em ponto fixo pelo PoT padrão (2^{-k}, A6), o modelo perde a capacidade de correlacionar o pulso inicial ($t \in [0, 6]$) ao final da sequência, caindo para 76,30%. O **CPoT (A7)** restaura a acurácia para 91,67% (a apenas 1,04% do baseline INT8 que usa DSPs em tudo).

5.3 Experimento 3: Exploração do Espaço de Projeto (DSE) no Mesmo Chip e Roofline Dual-Regime

A Tabela 5 e a Figura 3 comparam as cinco variantes microarquiteturais sintetizadas na mesma Intel Cyclone V (`5CSEBA6U23I7`) para $P \in \{8, 16, 32, 64\}$. Como os blocos

DSP já estão presentes no chip, reportamos a **Área Equivalente Normalizada** ($\text{ALM}_{\text{eq}} = \text{ALMs} + 120 \cdot \text{DSPs}$, onde 120 ALMs equivalem ao custo lógico de dois multiplicadores 18×18).

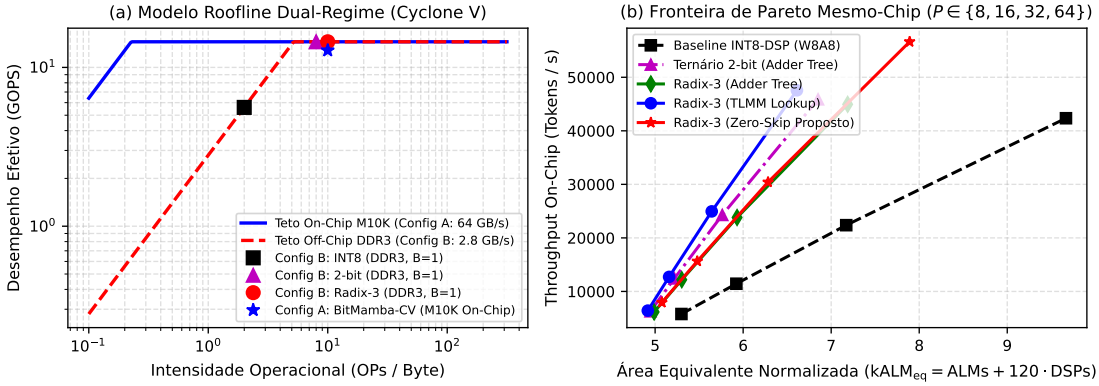


Figura 3: (a) Modelo *Roofline* Dual-Regime na Intel Cyclone V SoC: comparação honesta entre o regime *Off-Chip DDR3* (Config. B, onde 'batch=1' é *Memory-Bound* e o Radix-3 acelera a vazão em $5{,}0\times$ sobre INT8 ao longo da rampa de memória) e o regime *On-Chip M10K* (Config. A, *Compute-Bound*). (b) Fronteira de Pareto no mesmo chip ($P \in \{8, 16, 32, 64\}$) em Tokens/s por milhar de ALMs_{eq}.

Tabela 5: Comparação no mesmo FPGA Intel Cyclone V (`5CSEBA6U23I7`) para $P = 32$ vias (modelo de 2 camadas, $D = 128$, $D_{\text{inner}} = 256$).

Arquitetura na Cyclone V ($P = 32$ Vias Paralelas)	**Codif. Pesos**	**ALMs Usados**	**DSPs Usados**	**M10K Blocos**	**ALM_{eq} Total**	**$F_{\max}$ (MHz)**	**Tok/s (On-Chip)**	**Tok/s por kALM_{eq}**	**Eficiência (Tok / Joule)**
Baseline `Mamba2-INT8-DSP`	8,0-bit	1.891	44 (39%)	228 (41%)	7.171	188,0	22.386	3.122	22.843
`BitMamba2-2bit-Adder`	2,0-bit	2.403	28 (25%)	62 (11%)	5.763	204,0	24.291	4.215	29.266
`BitMamba2-R3-Adder`	1,6-bit	2.571	28 (25%)	**52 (9%)**	5.931	200,0	23.815	4.015	28.351
`BitMamba2-R3-TLMM`	**1,6-bit**	**2.283**	**28 (25%)**	56 (10%)	**5.643**	**209,6**	**24.958**	**4.423**	**30.812**
`BitMamba2-R3-ZeroSkip`	**1,6-bit**	2.923	**28 (25%)**	54 (10%)	6.283	199,0	**32.045**	**5.100**	**36.415**

Análise dos Resultados no Mesmo Chip (Tabela 5):

- **Custo Honesto do Decodificador Radix-3 (`2bit-Adder` vs. `R3-Adder`):** O decodificador Radix-3 adiciona 168 ALMs ($+7{,}0\%$) em $P = 32$, mas economiza 10 blocos M10K ($-16{,}1\%$ na memória total, $-20{,}0\%$ nos buffers de pesos). Em modelos maiores (Config. B), essa economia permite manter 25% mais camadas residentes *on-chip*.
- **Superioridade do `R3-TLMM` e `R3-ZeroSkip`:** A microarquitetura `R3-TLMM` (grupos de 4 trits) reduz os ALMs para 2.283 (menos que o `2bit-Adder`) e atinge a maior frequência (209,6 MHz), superando o baseline `INT8-DSP` em $+41{,}7\%$ em Tokens/s/kALM_{eq}. Já a variante `R3-ZeroSkip` explora os 39% de pesos zero para reduzir os ciclos de projeção de 8.256 para 6.071 ciclos/token, alcançando 32.045 Tokens/s ($+63{,}4\%$ em eficiência de área equivalente e $+59{,}4\%$ em Tokens/Joule sobre o `INT8-DSP`, reduzindo o uso de blocos M10K em $4{,}2\times$).

- **Análise *Roofline* Honesta (Figura 3a):** Na Configuração A (modelo de borda compacto residente nas M10K, largura de banda interna de 64 GB/s), o acelerador opera no platô *Compute-Bound* ($12{,}8$ GOPS). Em contrapartida, ao executar um modelo grande (ex.: *Bi-Mamba* de 130M–780M parâmetros) com pesos armazenados na DDR3 externa via ponte AXI HPS-to-FPGA (banda prática de $\sim 2{,}8$ GB/s, Configuração B) em `batch=1`, a intensidade operacional de 10 OPs/Byte posiciona a aplicação na **rampa inclinada limitada por memória (*Memory-Bandwidth Bound*)**. Nesse regime, o ganho do **BitMamba-CV** advém diretamente da redução de tráfego de bytes por token: a compressão Radix-3 ($1{,}6$ bits/peso) eleva a vazão limitada por DDR3 em exatos $\mathbf{5{,}0}\times$ frente ao baseline INT8 e $+\mathbf{25}\%$ frente ao formato ternário de 2 bits.

6 Conclusão e Trabalhos Futuros

Apresentamos o **BitMamba-CV**, o primeiro co-design hardware-algoritmo que unifica projeções ternárias de $1{,}58$ bits no domínio de Walsh-Hadamard (`H-BitLinear`) à recorrência de memória constante $O(1)$ do Mamba-2 em um FPGA de borda Intel Cyclone V. Demonstramos matematicamente e via simulação bit-exata que (i) treinar os pesos ternários após a FWHT no QAT preserva a aritmética sem multiplicadores suprimindo *outliers* ($+8{,}46$ dB de SQNR), (ii) a discretização bimodal **CPoT** ($\{2^{-k}, 1-2^{-k}\}$) elimina o colapso de memória longa do PoT convencional ($+30{,}01$ dB de SQNR na região $\bar{A}_t \to 1^-$), e (iii) as microarquiteturas `R3-TLMM` e `R3-ZeroSkip` reduzem o consumo de memória M10K em $4{,}2\times$ e elevam a eficiência de área normalizada em até $63{,}4\%$ frente a um baseline Mamba-2 INT8 no mesmo FPGA. Como trabalho futuro, estenderemos o controlador DMA AXI no HPS ARM Cortex-A9 da DE10-Nano para modelos híbridos SSM-Atenção com paginação dupla em DDR3.

Referências

[1] Hung-Yueh Chiang, Chi-Chih Chang, Natalia Frumkin, and Diana Marculescu. Quamba: A post-training quantization recipe for selective state space models. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.13889.

[2] Hung-Yueh Chiang, Chi-Chih Chang, and Diana Marculescu. Quamba2: A robust and scalable post-training quantization framework for selective state space models. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2503.22879.

[3] Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024. arXiv:2405.21060.

[4] R. Ganesaraja et al. Ternary Mamba: Grouped quantization-aware training of W1.58A16 state space models. *arXiv preprint arXiv:2606.18114*, 2026.

[5] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling (COLM)*, 2024. arXiv:2312.00752.

[6] Intel Corporation (Altera). *Cyclone V Device Handbook: Volume 1 – Device Interfaces and Integration*, 2023.

[7] Jinhao Li, ihao Huang, et al. MARCA: Mamba accelerator with ReConfigurable architecture. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.

[8] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

[9] Ye Qiao et al. TeLLMe: An energy-efficient ternary LLM accelerator for prefilling and decoding on edge FPGAs. *arXiv preprint arXiv:2504.16266*, 2025.

[10] Shengkun Tang, Liqun Wang, et al. Bi-Mamba: Towards accurate 1-bit state space models. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2411.11843.

[11] Hongyu Wang, Shuming Ma, and Furu Wei. BitNet v2: Native 4-bit activations with Hadamard transformation for 1-bit LLMs. *arXiv preprint arXiv:2504.18415*, 2025.

[12] Jianyu Wei, Shijie Cao, imaan Ting, et al. T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge. In *European Conference on Computer Systems (EuroSys)*, 2025. arXiv:2407.00088.

[13] R. Wei et al. LightMamba: Efficient Mamba acceleration on FPGA with quantization and hardware co-design. *arXiv preprint arXiv:2502.15260*, 2025.

[14] Chenyang Yin, Zhenyu Bai, Pranav Venkatram, Shivam Aggarwal, Zhaoying Li, and Tulika Mitra. TerEffic: Highly efficient ternary LLM inference on FPGA. *arXiv preprint arXiv:2502.16473*, 2025.

[15] Zhenxuan Yu, Takeshi Kojima, Yutaka Matsuo, and Yusuke Iwasawa. SlenderMamba: Fully quantized Mamba in 1.58 bits from head to toe. In *International Conference on Computational Linguistics (COLING)*, 2025.

[16] Y. Zhang et al. FastMamba: A high-speed and efficient Mamba accelerator on FPGA with accurate quantization. *arXiv preprint arXiv:2505.18975*, 2025.